

Bayesian Inference in Linear Regression: A Case
Study using Ironman Triathlon World
Championship Data
Faculty of Science and Mathematics, University of Niš

Anda Denić

August 23, 2023

ABSTRACT

This assignment explores the application of Bayesian inference in linear regression to predict the overall finish time of participants in the Ironman Triathlon World Championship. The study utilizes datasets from the 2018 and 2019 championships, with run time as the predictor variable and overall finish time as the response variable. The primary goal is to get familiar with Bayesian methods in the context of predictive modeling and compare it to classical statistics.

Each chapter of the assignment is accompanied by Python scripts that leverage popular libraries Pandas, NumPy, SciPy, and Matplotlib. These tools facilitate data manipulation, statistical computations, and visualization. The methodology involves using the 2018 dataset to inform the prior distribution, capturing underlying relationships between run time and finish time. The 2019 dataset is then split into training and testing sets, enabling the estimation of posterior distributions for model parameters using Bayesian techniques. The predictive performance of the Bayesian model is assessed on the test set, and the results are compared to those obtained through classical statistical methods using MAE and RMSE as metrics.

Through Bayesian inference, the study quantifies uncertainty in predictions by offering posterior distributions and credible intervals. This allows for a more comprehensive understanding of the potential range of outcomes. The comparison with classical statistics highlights the benefits of incorporating prior knowledge and accounting for uncertainty in predictions.

Contents

1	TERMINOLOGY AND DEFINITIONS	4
2	IRONMAN TRIATHLON DATASET	8
3	PREPROCESSING	11
3.1	Download the data	11
3.2	Take a look at the data	11
3.3	Data cleaning	12
3.4	Data visualizing	13
4	DECIDE ON PRIOR	16
4.1	Simple Linear Regression Assumptions	16
4.2	Bayes' Theorem for the Regression Model	16
4.3	Connection between Regularization and Choosing a Prior	21
5	CALCULATE POSTERIOR	23
6	COMPUTE PREDICTIVE DISTRIBUTIONS	28
7	EVALUATION	30
8	COMPARISON	32

List of Figures

3.1	Glance at our datasets	12
3.2	Distributions of 2018. and 2019. overall finish times	14
3.3	Dependence of run on finish time in 2018. and 2019.	15
4.1	Deciding on mean of $\alpha_{\bar{x}}$	19
4.2	Visualizing our prior for $\alpha_{\bar{x}}$	20
5.1	Prior and posterior distribution for parameter $\alpha_{\bar{x}}$	26
5.2	Prior and posterior distribution for parameter β	27

Chapter 1

TERMINOLOGY AND DEFINITIONS

Here is a list of key terms and concepts that are foundational our further analysis. This chapter will serve as a reference point for readers who may be less familiar with the terminology, helping them navigate and comprehend the subsequent content more effectively.

- **Population** is the entire group of objects or people the investigator wants information about. [?] For instance, the population might consist of all **Ironman** athletes. Usually we want to know some specific attribute about the population. Each member of the population has a number associated with it — for example, his/her overall finish time on triathlon. Then we can consider the **model population** to be the set of numbers for each individual in the real population. Our model population would be the set of overall finish times over the Ironman athletes. We want to learn about the distribution of the population. Specifically, we want information about the **population parameters**, which are numbers associated with the distribution of the population, such as the population mean, median, and standard deviation. Example of a population parameter is mean value of overall finish time over the whole population of Ironman triathletes. Often it is not feasible to get information about all the units in the population. The population may be too big, or spread over too large an area, or it may cost too much to obtain data for the complete population. So we do not know the parameters because it is infeasible to calculate them.
- **Sample** or **observation** is subset of the population. The investigator draws one sample from the population and gets information from the individuals in that sample. Sample statistics are calculated from sample data. They are numerical characteristics that summarize the distribution of the sample, such as the sample mean, median, and standard deviation. We use realizations of those statistics as approximations of parameters of

the population. In our example, we cannot calculate the mean value for finish times over the whole population, but we can calculate the mean value of given dataset. A statistic has a similar relationship to a sample that a parameter has to a population. However, the sample is known, so the statistic can be calculated.

- **(Statistical) inference** is process of drawing conclusions about a population on the basis of observations made on a sample from the population. [?]. Good inferences can be made if the sample is representative of the population as a whole! The distribution of the sample must be similar to the distribution of the population from which it came! Sampling bias, a systematic tendency to collect a sample which is not representative of the population, must be avoided. It would cause the distribution of the sample to be dissimilar to that of the population, and thus lead to very poor inferences.

We can make inference (conclusions) about properties of the population using different schools of statistical inference that have been established. Two most popular schools are frequentist and Bayesian. According to that we have frequentist and Bayesian inference.

- **Frequentist inference** is an approach to statistics based on a frequency view of probability in which it is assumed that it is possible to consider an infinite sequence of independent repetitions of the same statistical experiment. Significance tests, hypothesis tests and likelihood are the main tools associated with this form of inference. [?]
- **Bayesian inference** is an approach to inference based largely on Bayes' Theorem and consisting of the following principal steps:
 1. Obtain the likelihood, $f(x|\theta)$ describing the process giving rise to the data x in terms of the unknown parameters θ .
 2. Obtain the prior distribution, $f(\theta)$ expressing what is known about θ , prior to observing the data.
 3. Apply Bayes' theorem to derive the posterior distribution $f(\theta|x)$ expressing what is known about θ after observing the data.
 4. Derive appropriate inference statements from the posterior distribution. These may include specific inferences such as point estimates, interval estimates or probabilities of hypotheses.

This form of inference differs from the classical form of frequentist inference, the most important one being the use of the prior distribution which is not present in classical inference. Prior is the investigator's knowledge or belief about the parameter θ before seeing the data. Classical statistics uses only the likelihood.

- **Prior distribution** is probability distribution that summarize information about a random variable or parameter known or assumed at a given

time point, prior to obtaining further information from empirical data. Used almost entirely within the context of Bayesian inference. In any particular study a variety of such distributions may be assumed.

- **Posterior distribution** is probability distribution that summarize information about a random variable or parameter after, or a posteriori to, having obtained new information from empirical data. Used almost entirely within the context of Bayesian inference. [?]

Likelihood vs (Probability and Mass) Density Functions

Discrete Random Variable

Definition 1.0.1. Let X be our discrete random variable parameterized by a (possibly multivariate) unknown parameter θ with probability mass function $P_\theta(x)$. x is given single random observation of our random variable X . Likelihood function is a function of θ denoted as $L(\theta)$ or $L(\theta|x)$ or $L(\theta; x)$ and defined as:

$$L(\theta) = P_\theta(x) \quad (1.1)$$

Comment 1.0.1. When $P_\theta(x)$ is a function of θ and x is given, it is a likelihood function; but when $P_\theta(x)$ is a function of x and θ is given, it is a PMF (probability mass function).

Definition 1.0.2. If instead of single random observation in definition 1.0.1 we have a simple random sample $(x_1, \dots, x_n)$, likelihood is a function of θ defined as:

$$L(\theta) = \prod_{i=1}^n P_\theta(x_i) \quad (1.2)$$

Comment 1.0.2. When $P_\theta(x_1, \dots, x_n)$ is a function of θ and $(x_1, \dots, x_n)$ is given, it is a likelihood function; but when $P_\theta(x_1, \dots, x_n)$ is a function of $(x_1, \dots, x_n)$ and θ is given, it is a PMF (probability mass function).

Continuous Random Variable

Definition 1.0.3. Now, let X be our continuous random variable parameterized by a (possibly multivariate) unknown parameter θ with probability density function $f_\theta(x)$. x is given single random observation of our random variable X . Likelihood function is a function of θ denoted as $L(\theta)$ or $L(\theta|x)$ or $L(\theta; x)$ and defined as:

$$L(\theta) = f_\theta(x) \quad (1.3)$$

Comment 1.0.3. When $f_\theta(x)$ is a function of θ and x is given, it is a likelihood function; but when $f_\theta(x)$ is a function of x and θ is given, it is a PDF (probability density function).

Definition 1.0.4. If instead of single random observation in definition 1.0.3 we have simple random sample $(x_1, \dots, x_n)$ likelihood is a function of θ defined as:

$$L(\theta) = \prod_{i=1}^n f_{\theta}(x_i) \quad (1.4)$$

Comment 1.0.4. When $f_{\theta}(x_1, \dots, x_n)$ is a function of θ and $(x_1, \dots, x_n)$ is given, it is a likelihood function; but when $f_{\theta}(x_1, \dots, x_n)$ is a function of $(x_1, \dots, x_n)$ and θ is given, it is a PDF (probability density function).

Chapter 2

IRONMAN TRIATHLON DATASET

Triathlon



A triathlon is an endurance multisport race consisting of swimming, cycling, and running over various distances. Triathletes compete for fastest overall finish time, racing each segment sequentially with the time transitioning between the disciplines included. The Olympic triathlon consists of 1.5 km of swimming, 40 km of cycling, and 10 km of running.

Ironman Triathlon



One of the most popular ultra-triathlons is *Ironman*. The Ironman triathlon consists of 3.9 km of swimming, 181 km of cycling, and 42.2 km of running.

The Ironman Triathlon is a series of long-distance triathlon races organized by the World Triathlon Corporation (WTC). It is considered one of the most challenging endurance events in the world and consists of a 2.4-mile (3.86 km) swim, a 112-mile (180.25 km) bike ride, followed by a full marathon run of 26.2 miles (42.20 km). Participants must complete all three disciplines consecutively, with no breaks in between, within a specific time limit (usually 17 hours) to be considered an official finisher.

The race was originally conceived by U.S. Navy Commander John Collins in 1977 as a challenge to determine who were the fittest athletes: swimmers, bikers, or runners. The name "Ironman" comes from a conversation where participants in a previous race were discussing the difficulty of different endurance events, including the Waikiki Roughwater Swim (2.4 miles), the Around-Oahu Bike Race (115 miles), and the Honolulu Marathon (26.2 miles). They jokingly debated that combining all three events into one day would be the ultimate test of endurance—thus, the Ironman Triathlon was born.

The first Ironman Triathlon took place in Hawaii in 1978 and had 15 participants. It was held in the town of Kailua-Kona, and the event has been held there annually ever since. The Ironman World Championship, held in Kailua-Kona, is the most prestigious event in the series and is known as the pinnacle of the sport.

Over the years, the popularity of the Ironman Triathlon has grown exponentially, leading to the creation of numerous Ironman-branded races worldwide. Each year, thousands of athletes from all around the globe take part in Ironman events, pushing their physical and mental limits to complete the grueling course and earn the title of "Ironman" or "Ironwoman." The Ironman Triathlon embodies the principles of perseverance, determination, and the pursuit of personal

achievement

Why do we care about Ironman?



In the following homework assignment we will use data from Ironman World Championships from 2018. and 2019. More precisely, we will use only run and overall finish times for both years for athletes that finished the races.

Chapter 3

PREPROCESSING

Python scripts for this chapter are given in file named *preprocessing.py*.

3.1 Download the data

We will download results from 2018. and 2019. Ironman World Championship from obstri.com/races. Data sets are given as .csv files.

3.2 Take a look at the data

We will use *pandas* library in Python to take a look at the data.

```
1
2     # Setting the option to show all columns from DataFrame
3     pd.options.display.max_columns = None
4
5     # Read results from 2018. and 2019.
6     df_2018 = pd.read_csv(r'ironman_data\2018IRONMANWorldChampionship.csv',
7                           delimiter=';')
8     df_2019 = pd.read_csv(r'ironman_data\2019IRONMANWorldChampionship.csv',
9                           delimiter=';')
10
11    # Take a look at the first five and last five rows of the data
12    print(df_2018.head())
13    print(df_2019.head())
14    print(df_2018.tail())
15    print(df_2019.tail())
```

When we run the code above we get following results. On picture [3.1a](#) we can see first five rows and on the picture [3.1b](#) we can see last five rows of each dataset.

	First	Last	Nation	Division	Status	Swin	T1	Bike	\
0	Philippine	Cheveldor	CH	M40-44	DNF	1:11:13	2:33	---	---
1	Hariko	Kirihara	JP	F45-49	DNF	1:15:21	9:45	---	---
2	Konstantin	Mentis	SR	M55-59	DNF	2:32:45	---	---	---
3	Jacob	Dolecki	US	M18-24	DNF	1:07:27	3:56	---	---
4	Tony	O'Keeffe	CA	M55-59	DNF	1:04:00	3:51	5:07:16	

	T2	Run	Overall
0	---	---	---
1	---	---	---
2	---	---	---
3	---	---	---
4	3:34	---	---

	First	Last	Nation	Division	Status	Swin	T1	Bike	T2	\
0	Dean	Paxson	US	M75-79	DNF	1:46:52	13:05	8:23:11	10:33	
1	Ka Hung	Tsang	HK	F18-24	DNF	1:12:02	2:43	7:28:26	---	---
2	Jordan	Blanco	GB	F45-49	DNF	1:12:23	5:07	5:32:43	9:14	
3	Robert	Plant	US	M75-79	DNF	1:28:55	4:54	7:12:40	---	---
4	Håkan	Börjesson	SE	M75-79	DNF	1:54:34	9:53	8:13:50	14:10	

	Run	Overall
0	---	---
1	---	---
2	---	---
3	---	---
4	---	---

(a) first five rows of datasets

	First	Last	Nation	Division	Status	Swin	T1	Bike	\
2363	Marcus	Cook	US	M45-49	Finish	1:53:28	11:03	7:38:00	
2364	Xiangdong	Wen	CN	M65-69	Finish	1:55:16	8:51	7:47:31	
2365	Hiroe	Inada	JP	M85-89	Finish	1:51:25	12:08	8:02:40	
2366	George	Hudson	US	M75-79	Finish	2:13:39	13:20	7:34:06	
2367	Guillermo	Franco	MX	M40-44	Finish	1:15:19	7:11	7:16:52	

	T2	Run	Overall
2363	11:52	6:56:45	16:51:00
2364	10:26	6:51:17	16:53:21
2365	19:19	6:28:17	16:53:49
2366	11:19	6:41:41	16:56:05
2367	17:17	8:00:16	16:56:55

	First	Last	Nation	Division	Status	Swin	T1	\
2016	Yoko	Ishibashi	JP	F65-69	Finish	1:40:51	8:14	
2017	Mike	Anthony	GB	M65-69	Finish	1:29:17	10:49	
2018	Cathy	Beavers Hamner	US	F65-69	Finish	1:45:30	10:25	
2019	Paul	Whitehouse	US	M70-74	Finish	2:06:01	11:49	
2020	Edgardo	Piasentini	AR	M65-69	Finish	1:42:27	6:17	

	Bike	T2	Run	Overall
2016	8:21:43	9:53	6:23:49	16:44:30
2017	7:46:49	14:48	7:04:37	16:46:20
2018	8:25:01	10:15	6:18:52	16:50:03
2019	7:51:20	11:20	6:29:34	16:50:04
2020	7:42:09	8:39	7:13:49	16:53:21

(b) Last five rows of datasets

Figure 3.1: Glance at our datasets

3.3 Data cleaning

From 3.1a we see that there are athletes that have missing times. That is because they didn't start the race or they didn't finish the race in given time (which is 17 hours). Their **Status** column is **DNS** or **DNF**. Because we will use finish times of athletes that finished the race, we first need to clean the data. If athlete finished the race his/her **Status** is **Finish**. We will use this condition to clean the incomplete data. This is the part of pre-processing.

```

1 # Look only at athletes that finished triathlon
2     df_2018 = df_2018[df_2018['Status'] == 'Finish']
3     df_2019 = df_2019[df_2019['Status'] == 'Finish']
4
5     # Make lists out of columns that represent run and overall times in format HH:MM:SS
6     overall_list_2018 = df_2018['Overall'].tolist()
7     run_list_2018 = df_2018['Run'].tolist()
8     overall_list_2019 = df_2019['Overall'].tolist()
9     run_list_2019 = df_2019['Run'].tolist()

```

Because we will use only run and overall finish times for our inference, we extract that data in four output lists. Times are given in unwanted format *HH : MM : SS* and we want regular float numbers of hours. We make a simple method `convert_to_hours` to do that:

```

1 def convert_to_hours(input_list: list):
2     """
3     Method takes a list of times in format HH:MM:SS
4     and returns corresponding numpy array of float number of hours
5     """

```

```

6         output_list = []
7         for time_str in input_list:
8             hours, minutes, seconds = map(float, time_str.split(':'))
9             total_hours = hours + (minutes / 60) + (seconds / 3600)
10            output_list.append(total_hours)
11        output_list = np.array(output_list)
12        return output_list

```

We use method above for all four lists and out of the outputs make unique DataFrame of relevant data and export it to csv file we will use later.

```

1        # Change the format to float number o hours in lists above
2        overall_2018 = convert_to_hours(overall_list_2018)
3        run_2018 = convert_to_hours(run_list_2018)
4        overall_2019 = convert_to_hours(overall_list_2019)
5        run_2019 = convert_to_hours(run_list_2019)
6
7        # Export relevant data as a csv file
8        df_output_2018 = pd.DataFrame({
9            "run_2018": run_2018,
10           "overall_2018": overall_2018
11        })
12        df_output_2018.to_csv("ironman_data\data_2018_lin_reg.csv")
13        df_output_2019 = pd.DataFrame({
14            "run_2019": run_2019,
15            "overall_2019": overall_2019
16        })
17        df_output_2019.to_csv("ironman_data\data_2019_lin_reg.csv")
18

```

3.4 Data visualizing

Let's visualize distributions of overall finish times for both years using .csv files we made earlier using matplotlib.pyplot library in Python.

```

# Visualise distribution of overall finish times in both years
fig, ax = plt.subplots()
ax.hist(overall_2018, bins=np.linspace(7, 17, 11), alpha=0.4, color='deeppink',
        label='2018. overall finish times')
ax.hist(overall_2019, bins=np.linspace(7, 17, 11), alpha=0.4, color='orange',
        label='2019. overall finish times')
ax.legend()
ax.set_title('Histogram of 2018. and 2019. overall finish times (hrs)')
ax.set_xlabel('2018. and 2019. overall finish times (hrs)')
ax.set_ylabel('Frequency')
plt.show()

```

We get the following histogram:

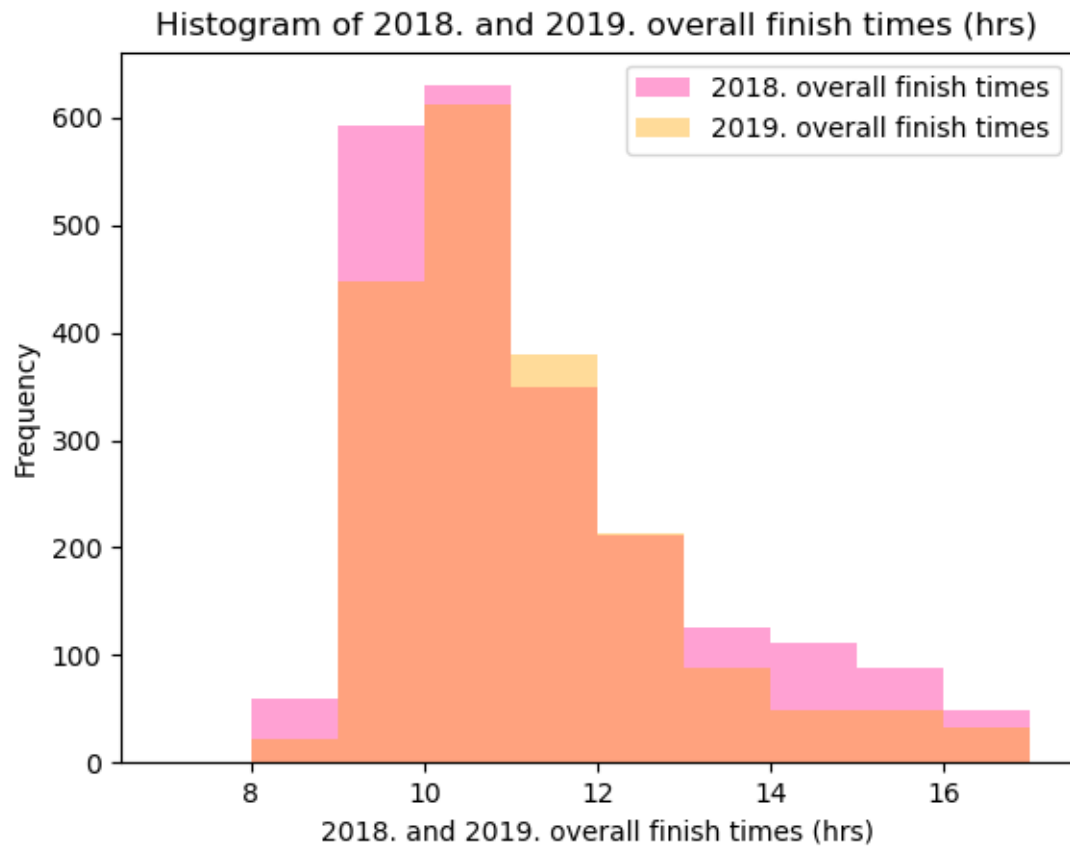


Figure 3.2: Distributions of 2018. and 2019. overall finish times

Now let's visualize how overall finish times depends on run portion.

```
# Visualise how overall finish times depends on run portion
plt.figure(figsize=(10, 6))
plt.scatter(run_2018, overall_2018, s=.5, alpha=.4, c='deeppink', label='2018. Ironman')
plt.scatter(run_2019, overall_2019, s=.5, alpha=.4, c='red', label='2019. Ironman')
plt.legend()
plt.show()
```

We get the following histogram:

From looking at the plot 3.3 we can say that it is okay assume that overall finish time linearly depends of run portion.

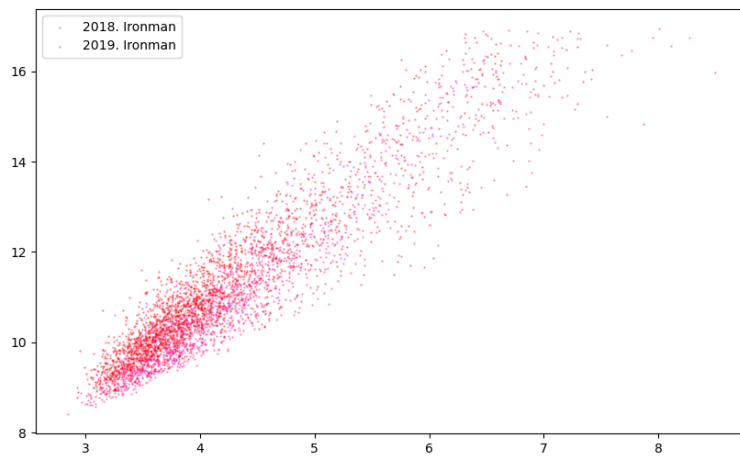


Figure 3.3: Dependence of run on finish time in 2018. and 2019.

Chapter 4

DECIDE ON PRIOR

4.1 Simple Linear Regression Assumptions

We have results from 2018. Ironman triathlon and we want to use it to analyse how athlete running time affects his/her overall finish time. Dataset has results of 2218 athletes that finished the race in given time frame.

Our goal here is to use run time, denoted as variable x and called the predictor variable, to help us predict overall finish time denoted as variable y and called the response variable.

$D = \{(x_i, y_i) | i = 1..n\}$, $n = 2218$ is our data set.

We assume that variables are related by a linear relationship.

The simple linear regression assumptions are:

- **(Mean assumption)** The conditional mean of y given x is an unknown linear function of x .

$$\mu_{y|x} = \alpha_{\bar{x}} + \beta(x - \bar{x})$$

where β is the unknown slope and $\alpha_{\bar{x}}$ is unknown intercept of the vertical line $x = \bar{x}$. Both β and $\alpha_{\bar{x}}$ are random variables in Bayes' inference.

- **(Error assumption)** Observation equals mean plus error, which is normally distributed with mean 0 and known variance σ^2 . All errors e_i have equal variances.

$$y_i = \alpha_{\bar{x}} + \beta(x - \bar{x}) + e_i$$

In formula above β , $\alpha_{\bar{x}}$ and e_i , for all $i = 1..n$ are random variables.

- **(Independence assumption)** The errors for all of the observations are independent of each other.

4.2 Bayes' Theorem for the Regression Model

If we chose normal priors for our parameters β and $\alpha_{\bar{x}}$, then posterior distributions are normal as well. Parameters of the posterior distribution can be found

from the simple updating rules shown in table 4.1 below:

	prior	posterior
β	$\sim \mathcal{N}(m_\beta, s_\beta^2)$	$\sim \mathcal{N}(m'_\beta, s'^2_\beta)$
$\alpha_{\bar{x}}$	$\sim \mathcal{N}(m_{\alpha_{\bar{x}}}, s_{\alpha_{\bar{x}}}^2)$	$\sim \mathcal{N}(m'_{\alpha_{\bar{x}}}, s'^2_{\alpha_{\bar{x}}})$

Table 4.1: Normal prior and posterior distribution for linear regression parameters

In table 4.1 we choose parameters for prior. In other words, we choose constants m_β , s_β , $m_{\alpha_{\bar{x}}}$ and $s_{\alpha_{\bar{x}}}$. In following section we will describe how to choose them in a smart way. Parameters in posterior distributions m'_β , s'_β , $m'_{\alpha_{\bar{x}}}$ and $s'_{\alpha_{\bar{x}}}$ are computed using following update rules:

$$\begin{aligned}\frac{1}{(s'_\beta)^2} &= \frac{1}{s_\beta^2} + \frac{SS_x}{\sigma^2} \\ m'_\beta &= \frac{\frac{1}{s_\beta^2}}{\frac{1}{(s'_\beta)^2}} \times m_\beta + \frac{\frac{SS_x}{\sigma^2}}{\frac{1}{(s'_\beta)^2}} \times B. \\ \frac{1}{(s'_{\alpha_{\bar{x}}})^2} &= \frac{1}{s_{\alpha_{\bar{x}}}^2} + \frac{n}{\sigma^2} \\ m'_{\alpha_{\bar{x}}} &= \frac{\frac{1}{s_{\alpha_{\bar{x}}}^2}}{\frac{1}{(s'_{\alpha_{\bar{x}}})^2}} \times m_{\alpha_{\bar{x}}} + \frac{\frac{n}{\sigma^2}}{\frac{1}{(s'_{\alpha_{\bar{x}}})^2}} \times A_{\bar{x}}.\end{aligned}$$

Above we used intermediate results:

$$\begin{aligned}\bar{x} &= \frac{\sum_{i=1}^n x_i}{n} \\ \bar{y} &= \frac{\sum_{i=1}^n y_i}{n} \\ SS_x &= \sum_{i=1}^n (x_i - \bar{x})^2 \\ B &= \frac{SS_{xy}}{SS_x} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} \\ A_{\bar{x}} &= \bar{y}\end{aligned}$$

How to choose normal priors for β and $\alpha_{\bar{x}}$?

Python scripts for this section is given as Jupyter Notebook and file is named *choosing_prior*.

Prior mean for $\alpha_{\bar{x}}$

First we will decide our prior mean for $\alpha_{\bar{x}}$. To do that we just need to decide what we believe the mean value of the y is. That will be $m_{\alpha_{\bar{x}}}$. Using `numpy` library in Python, we can calculate $\bar{y}$ using `mean` method of a numpy array y_{2018} .

```
1 import numpy as np
2
3 y_mean = np.mean(y_2018)
4 print(y_mean)
```

We got 11.24 for y_mean , so our prior belief is:

$$m_{\alpha_{\bar{x}}} = 11.24$$

Prior standard deviation for $\alpha_{\bar{x}}$

To decide on standard deviation for $\alpha_{\bar{x}}$ we have to think of the points above and below we consider to be upper and lower bounds of the possible values of y . By dividing the difference by 6 we get $s_{\alpha_{\bar{x}}}$, our prior standard deviation of $\alpha_{\bar{x}}$. We will do that by looking at the plot that shows dependence of run time on finish time in 2018. Ironman.

```
1 plt.figure()
2 plt.scatter(x_2018, y_2018, color='deeppink', s=1)
3 plt.xlabel('Run time')
4 plt.ylabel('Overall finish time')
5 plt.title('Dependence of run time on overall time on 2018. Ironman')
6 plt.plot([2, 9], [y_mean, y_mean], '--', linewidth=1, color='teal',
7          label=r'$y = \bar{y}$')
8 plt.plot([x_mean, x_mean], [8, 16], '--', linewidth=1, color='hotpink',
9          label=r'$x = \bar{x}$')
10 plt.legend()
11 plt.show()
```

By running the script above we get the plot

From looking at the plot we decide that lower bound is 9.5 and upper bound is 13.5. The script for calculating standard deviation is:

```
1 lower_bound_alpha_x, upper_bound_alpha_x = 9.5, 13.5
2 std_prior_alpha_x = (upper_bound_alpha_x - lower_bound_alpha_x) / 6
3 print(std_prior_alpha_x)
```

After running the script we got `std_prior_alpha_x = 0.6666666666666666`, so for standard deviation for $\alpha_{\bar{x}}$ we will use:

$$s_{\alpha_{\bar{x}}} = 0.67$$

To visualize our prior for $\alpha_{\bar{x}}$ we use the following script:

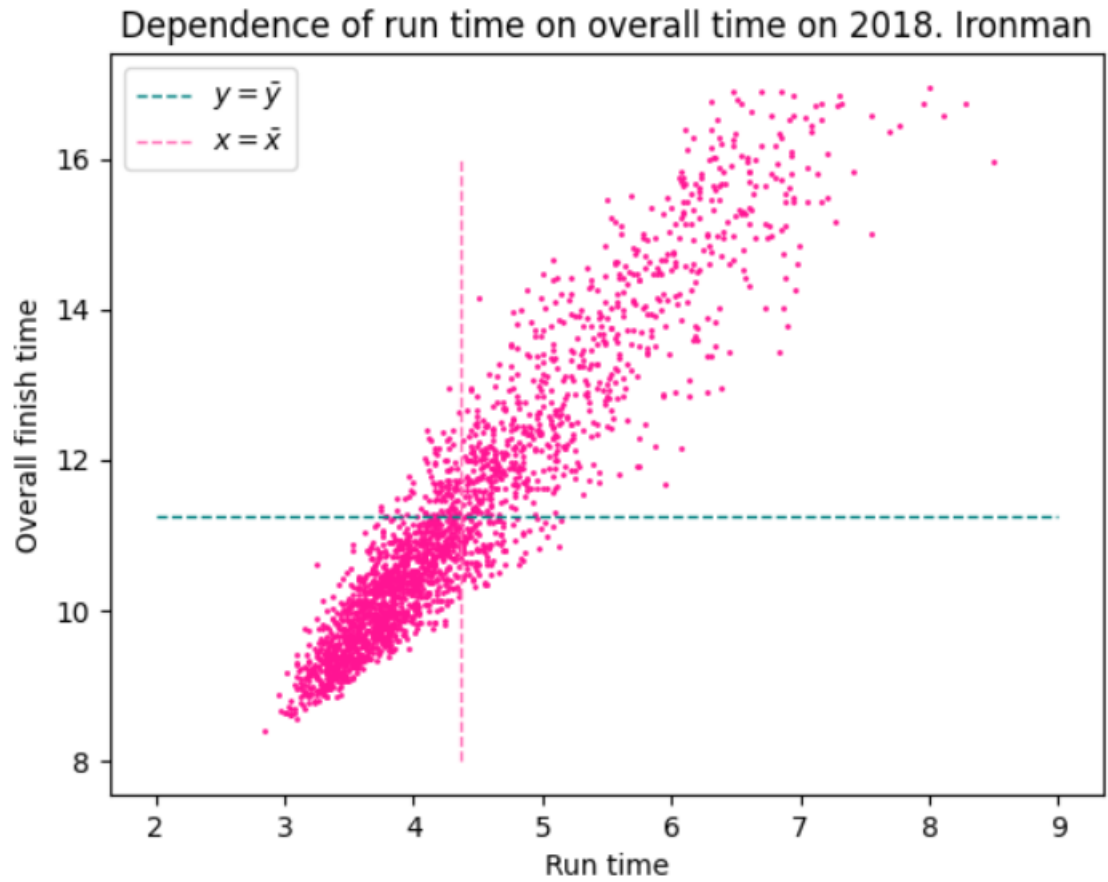


Figure 4.1: Deciding on mean of $\alpha_{\bar{x}}$

```

1 plt.figure()
2 plt.scatter(x_2018, y_2018, color='pink', s=.7)
3 plt.xlabel('Run time')
4 plt.ylabel('Overall finish time')
5 plt.title('Dependence of run time on overall time on 2018. Ironman')
6
7 plt.plot([x_mean, x_mean], [8, 16], '--', linewidth=1, color='hotpink',
8         label=r'$x = \bar{x}$')
9 plt.plot([2, 9], [y_mean, y_mean], '--', linewidth=1.5, color='teal',
10        label=r'prior mean for $\alpha_{\bar{x}}$')
11
12 plt.plot([2, 9], [11.24 + std_prior_alpha_x, 11.24 + std_prior_alpha_x], '--',
13        linewidth=1.5, color='greenyellow',

```



```

14     label=r'1 std from mean')
15     plt.plot([2, 9], [11.24 - std_prior_alpha_x, 11.24 - std_prior_alpha_x], '--',
16               linewidth=1.5, color='greenyellow', label=r'1 std from mean')
17     plt.plot([2, 9], [11.24 + 3*std_prior_alpha_x, 11.24 + 3*std_prior_alpha_x], '--',
18               linewidth=1, color='greenyellow', label=r'3 stds from mean')
19     plt.plot([2, 9], [11.24 - 3*std_prior_alpha_x, 11.24 - 3*std_prior_alpha_x],
20               '--', linewidth=1, color='greenyellow', label=r'3 stds from mean')
21
22     plt.legend()
23     plt.show()

```

We get the following visualization:

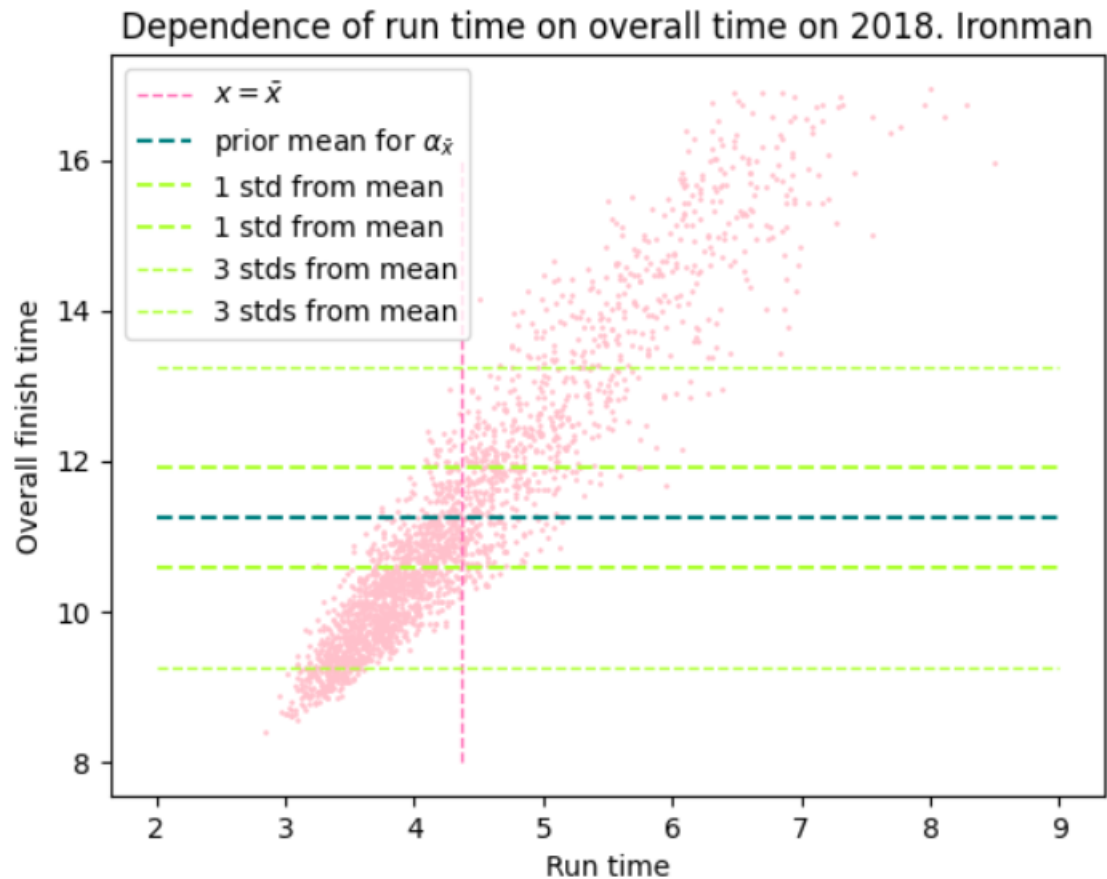


Figure 4.2: Visualizing our prior for $\alpha_{\bar{x}}$

Prior for β

Usually we are more interested in the slope β . Sometimes we want to determine if it could be 0. Therefore we may choose $m_\beta = 0$ as the prior mean for β . In other cases, just like in ours, we have prior belief about the slope from previous data. Then we think of the upper and lower bounds of the effect of an increase in x of one unit on y . Divide the difference by 6 to get s_β , our prior standard deviation of β . Then, we would use the normal($m_\beta, (s_\beta)^2$) that matches that prior belief.

For m_β we will use estimation of parameter β from classical statistics and data from 2018: $\frac{\bar{xy} - \bar{x}\bar{y}}{x^2 - \bar{x}^2}$ and for s_β we use visualisation ?? and we get:

$$m_\beta = 1.85$$
$$s_\beta = \frac{1}{3}$$

4.3 Connection between Regularization and Choosing a Prior

Regularization in linear regression and choosing a prior for parameters in Bayesian linear regression are related concepts, both aiming to address overfitting and improve the generalization performance of the model. However, they approach the problem from different perspectives and methodologies.

Regularization in Linear Regression: Regularization techniques, such as Ridge (L2) and Lasso (L1) regularization, are used in classical (frequentist) linear regression to prevent overfitting by adding a penalty term to the loss function that the model minimizes. This penalty term discourages large parameter values, effectively constraining the model's complexity. In Ridge regression, the penalty term is the sum of squared magnitudes of the coefficients, which encourages smaller coefficients but doesn't necessarily push them to exactly zero. In Lasso regression, the penalty term is the sum of absolute values of the coefficients, leading to some coefficients being exactly zero and thus performing feature selection.

Choosing a Prior for Parameters in Bayesian Linear Regression: In Bayesian linear regression, you treat the model parameters as random variables with probability distributions called priors. Priors represent your beliefs or assumptions about the possible values of the parameters before observing any data. When new data is observed, the prior is combined with the likelihood of the data given the parameters to form the posterior distribution over the parameters. A normal (Gaussian) prior is often used in Bayesian linear regression as it is mathematically convenient and has some nice properties. A flat prior, on the other hand, is a non-informative prior that doesn't impose any constraints or assumptions on the parameter values.

Connection between Regularization and Priors: The connection between regularization and choosing a prior in Bayesian linear regression is that certain

regularization methods used in frequentist linear regression can be interpreted as imposing specific forms of priors in the Bayesian framework.

Ridge Regression and Normal Prior: Ridge regression can be seen as equivalent to imposing a normal (Gaussian) prior with mean zero on the parameters. The L2 penalty term in Ridge regression penalizes large parameter values, which corresponds to placing more probability mass around smaller parameter values in the Bayesian framework.

Lasso Regression and Laplace Prior: Lasso regression can be thought of as imposing a Laplace prior (double-exponential) on the parameters. The L1 penalty term in Lasso encourages sparsity by driving some coefficients to exactly zero, which aligns with the sparse nature of the Laplace distribution.

In summary, while regularization techniques and choosing priors in Bayesian linear regression address overfitting and control model complexity, they do so through different mathematical mechanisms. Regularization directly adds a penalty term to the loss function, whereas Bayesian priors introduce uncertainty through probability distributions over the parameters. The connection lies in how certain regularization methods can be interpreted as imposing specific forms of priors in a Bayesian context, revealing a conceptual link between the two approaches.

Chapter 5

CALCULATE POSTERIOR

Python scripts

We will use update rules from previous chapter to write Python method that returns mean and standard deviation of normal posterior distribution of parameters β and $\alpha_{\bar{x}}$. This method is given in *compute_posterior.py* file.

```
1     import numpy as np
2     import matplotlib.pyplot as plt
3     from scipy.stats import norm
4     from scipy.stats import t
5
6
7     def posterior_for_normal_prior(x: np.ndarray, y: np.ndarray,
8                                   m_alpha_x: float, s_alpha_x: float,
9                                   m_beta: float, s_beta: float):
10
11         """
12         Returns distribution parameters of slope beta and x_bar intersect alpha_x
13         for simple linear regression model:
14         y = beta * (x - x_bar) + alpha_x
15         given the data (x - predictor, y - response variable) and assuming that
16         priors for parameters are normal with distributions:
17         beta ~ (m_beta, s_beta^2) and
18         alpha_x ~ (m_alpha_x, s_alpha_x^2),
19         Posterior distributions are normal as well with distribution:
20         beta ~ (m_beta_prim, s_beta_prim_squared) and
21         alpha_x ~ (m_alpha_x_prim, s_alpha_x_prim_squared)
22         """
23
24         # Calculating intermediate results
25         x_mean = np.mean(x)
26         SSx = np.sum((x - x_mean) ** 2)
27         n = len(y)
```

```

26
27     B = (np.mean(x * y) - np.mean(x) * np.mean(y)) /
28         (np.mean(x ** 2) - (np.mean(x) ** 2))
29     y_intercept = np.mean(y) - B * np.mean(x)
30     SSR = np.sum((y - B * x - y_intercept) ** 2) # Sum of Squared Residuals
31     sigma_squared = SSR / (n - 2)
32     Ax_bar = np.mean(y)
33
34     # Calculating output
35     s_beta_prim_squared = 1 / (1 / (s_beta ** 2) + SSx / sigma_squared)
36     m_beta_prim = (s_beta_prim_squared / (s_beta ** 2)) * m_beta + \
37         ((SSx * s_beta_prim_squared) / sigma_squared) * B
38     s_alpha_x_prim_squared = 1 / ((1 / s_alpha_x ** 2) + (n / sigma_squared))
39     m_alpha_x_prim = (s_alpha_x_prim_squared / (s_alpha_x ** 2)) * m_alpha_x + \
40         ((n * s_alpha_x_prim_squared) / sigma_squared) * Ax_bar
41     return m_alpha_x_prim, s_alpha_x_prim_squared,
42         m_beta_prim, s_beta_prim_squared,
43         sigma_squared

```

We will split our data from 2019. Ironman into two subsets: train and test. Splitting the data into train and test sets is a fundamental practice in the development of predictive statistical models, especially in machine learning. This approach serves several purposes, the primary one being evaluating the model. The reason is to evaluate how well the model performs on new, unseen data. By training the model on one subset (the training set) and then evaluating its performance on another subset (the test set), we can estimate how well the model will generalize to new data it has never encountered before. For splitting we will use `train_test_split` function from `sklearn`.

We will use `compute_posterior` method in main file:

```

1     from sklearn.model_selection import train_test_split
2     from compute_posterior import *
3
4     # Split the data from 2019. on train and test sets
5     x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
6         random_state=42)
7
8     # Compute posterior distributions for beta and alpha_x
9     mean_posterior_alpha_x, std_posterior_alpha_x_squared,
10     mean_posterior_beta, std_posterior_beta_squared,
11     std_squared = \
12     posterior_for_normal_prior(x_train, y_train, mean_prior_alpha_x,
13         std_prior_alpha_x, mean_prior_beta, std_prior_beta)
14

```

Outputs are parameters of posterior distributions:

$$m'_{\alpha_x=11.13}$$

$$s'^2_{\alpha_{\bar{x}}}=0.0001$$

$$m'_{\beta} = 1.76$$

$$s'^2_{\beta} = 0.02$$

We can conclude that our posterior distributions are:

$$\alpha_{\bar{x}} \sim \mathcal{N}(11.13, 0.0001^2)$$

$$\beta \sim \mathcal{N}(1.76, 0.02^2)$$

Visualize prior and posterior distributions

We will write Python script to visualize prior and posterior for both parameters.
This is done in file *visualization.py*.

```

1     import matplotlib.pyplot as plt
2     import numpy as np
3     from scipy.stats import norm
4
5
6     def plot_prior_and_posterior(mean_prior: float, std_prior: float,
7                                   mean_posterior: float, std_posterior_squared: float,
8                                   label_of_fig: str, name_of_fig:str):
9         """
10        Method is used for visualising prior and posterior distribution
11        if prior is normal(mean_prior, std_prior^2)
12        and posterior is normal(mean_posterior, std_posterior_squared).
13        Figure will be saved in plots folder.
14        """
15        left, right = min(mean_prior, mean_posterior), max(mean_prior, mean_posterior)
16        max_std = max(std_prior, std_posterior_squared ** .5)
17        start, finish = left - 3*max_std, right + 3*max_std
18        x_plot = np.linspace(start, finish, 500)
19        y_prior = norm.pdf(x_plot, mean_prior, std_prior)
20        y_posterior = norm.pdf(x_plot, mean_posterior, std_posterior_squared ** .5)
21
22        plt.figure(figsize=(8, 6))
23        plt.plot(x_plot, y_prior, linestyle='--', label='prior distribution',
24                 color='deeppink')
25        plt.plot(x_plot, y_posterior, label='posterior distribution',
26                 color='deeppink')
27        plt.xlabel(r'value of a parameter')
28        plt.ylabel('probability density')
29        plt.title(label_of_fig)
30        plt.legend()

```

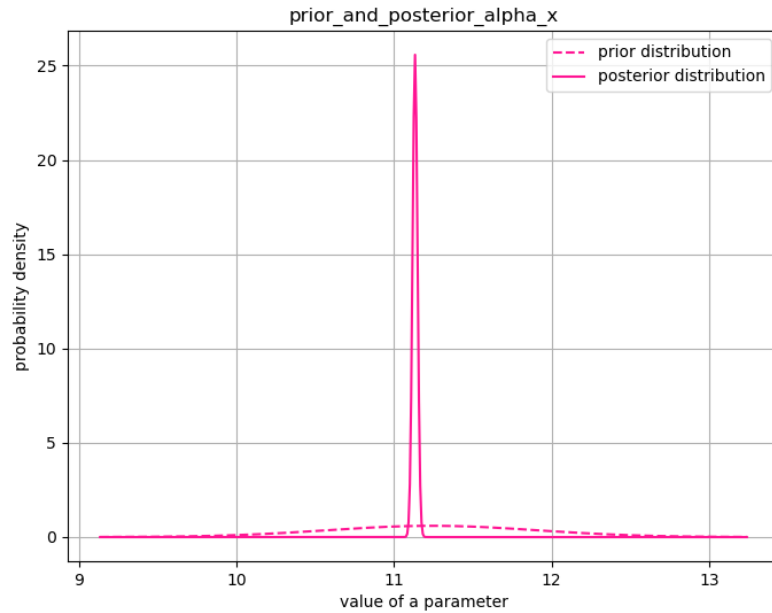


Figure 5.1: Prior and posterior distribution for parameter $\alpha_{\bar{x}}$

```

31     plt.grid(True)
32     plt.savefig('plots/' + name_of_fig + '.png')
33     plt.show()
34

```

When we use this function in main, we get the following result:

```

1     from visualization import *
2
3
4     # Visualize prior and posterior distributions for both parameters
5     plot_prior_and_posterior(mean_prior_alpha_x, std_prior_alpha_x,
6                               mean_posterior_alpha_x, std_posterior_alpha_x_squared,
7                               r'Prior and posterior distriburions of $\alpha_{\bar{x}}$',
8                               'prior_and_posterior_alpha_x')
9     plot_prior_and_posterior(mean_prior_beta, std_prior_beta,
10                               mean_posterior_beta, std_posterior_beta_squared,
11                               r'Prior and posterior distributions of $\beta$',
12                               'prior_and_posterior_beta')

```

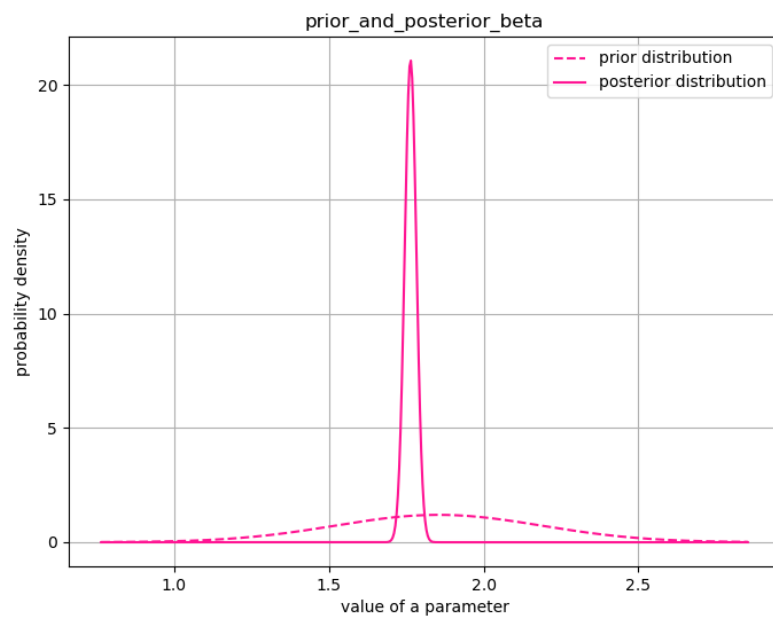


Figure 5.2: Prior and posterior distribution for parameter β

Chapter 6

COMPUTE PREDICTIVE DISTRIBUTIONS

Making predictions of future observations for specified x values is one of the main purposes of linear regression modelling. Often, after we have established from the data that there is a linear relationship between the explanatory variable x and the response variable y , we want to use that relationship to make predictions of the next value y_{n+1} , given the next value of the explanatory variable x_{n+1} . We can make better predictions using the value of the explanatory variable than without it. The best prediction for y_{n+1} given x_{n+1} will be:

$$\tilde{y}_{n+1} = \hat{\alpha}_{\bar{x}} + \hat{\beta} \times (x_{n+1} - \bar{x}),$$

where $\hat{\beta}$ is the slope estimate and $\hat{\alpha}_{\bar{x}}$ is the estimate of the intercept of the line $x = \bar{x}$.

How good is the prediction? There are two sources of uncertainty. First, we are using the estimated values of the parameters in the prediction, not the true values, which are unknown. We are considering the parameters to be random variables and have found their posterior distribution in the previous section. Second, the new observation y_{n+1} contains its own observation error e_{n+1} , which will be independent of all previous observation errors. The predictive distribution of the next observation y_{n+1} given the value x_{n+1} and the data accounts for both sources of uncertainty. It is denoted $f(y_{n+1} | x_{n+1}, \text{data})$ and is found by Bayes' theorem. The result is following:

$$\text{normal}(m'_y, (s'_y)^2)$$

where $m'_y = m'_\mu$, and $(s'_y)^2 = (s'_\mu)^2 + \sigma^2$. Thus the predictive mean of the next observation y_{n+1} taken at x_{n+1} is the posterior mean of $\mu_{n+1} = \alpha_{\bar{x}} + \beta(x_{n+1} - \bar{x})$, and the predictive variance of y_{n+1} is the posterior variance of $\mu_{n+1} = \alpha_{\bar{x}} + \beta(x_{n+1} - \bar{x})$ plus the observation variance σ^2 . Thus both sources of uncertainty have been allowed for in the predictive distribution.

We want to predict finish times when observations are run times from our test set. To calculate parameters for predictive observations we will write Python function given in *predictions.py* file.

```

1     import numpy as np
2
3
4     def compute_predictive_distributions(x: np.ndarray, x_bar: float,
5         m_alpha_x_prim: float, sigma_alpha_x_prim_squared: float,
6         m_beta_prim: float, sigma_beta_prim_squared: float,
7         sigma_squared: float):
8         """
9         For given numpy array of new observations x, method compute parameters
10        (mean and std) of predictive distribution
11        for each observation.
12        Method returns two numpy arrays where the first one collects means and
13        the second one
14        collects squared standard deviations.
15        """
16        m_y_prim = m_alpha_x_prim + (x - x_bar) * m_beta_prim
17        s_y_prim_squared = sigma_squared + sigma_alpha_x_prim_squared +
18        sigma_beta_prim_squared * (x - x_bar) * (x - x_bar)
19        return m_y_prim, s_y_prim_squared

```

We will use `compute_posterior` method in main file:

```

1     # Compute parameters for normal predictive distribution for future observations
2     x_mean = np.mean(x)
3     mean_pred, std_pred_squared = compute_predictive_distributions(x_test, x_mean,
4         mean_posterior_alpha_x, std_posterior_alpha_x_squared,
5         mean_posterior_beta, std_posterior_beta_squared,
6         std_squared)
7
8     y_sample_of_predictions = np.random.normal(mean_pred, std_pred_squared ** .5)

```

`y_sample_of_predictions` is numpy array that collects predictions for our test set.

Chapter 7

EVALUATION

RMSE (Root Mean Squared Error) and MAE (Mean Absolute Error) are both commonly used evaluation metrics for regression problems. They help us quantify the accuracy of a regression model's predictions by measuring the difference between the predicted values and the actual target values. Each metric has its own characteristics that make it suitable for different scenarios.

Root Mean Squared Error (RMSE): RMSE is a widely used metric for evaluating the accuracy of a regression model. It measures the average magnitude of the errors between predicted and actual values, while giving more weight to larger errors due to the squared term. The formula for RMSE is as follows:

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

RMSE penalizes larger errors more heavily, which can make it sensitive to outliers. This characteristic can be useful in cases where you want to prioritize accurate predictions and larger errors are particularly undesirable.

Mean Absolute Error (MAE): MAE is another commonly used metric for assessing regression models. Unlike RMSE, it calculates the average of the absolute differences between predicted and actual values. The formula for MAE is as follows:

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

MAE treats all errors equally and does not give extra weight to larger errors like RMSE does. This can make it more robust to the presence of outliers in the data. If the dataset contains outliers that you want the model to be less affected by, MAE might be a more suitable choice.

It's also worth noting that both metrics have the same unit as the target variable, making them easy to interpret in real-world terms. When comparing models or making decisions, it's a good practice to evaluate them using multiple metrics and consider the specific characteristics of your problem and data.

We will use both errors to evaluate our model. We can easily do this in our main file:

```
1     # Evaluation
2     mae = np.mean(np.abs(y_sample_of_predictions - y_test)) # mean absolute error
3     rmse = (np.mean((y_sample_of_predictions - y_test)**2)) ** .5 # root mean squared error
4     print(f'mae = {mae}')
5     print(f'rmse = {rmse}')
6
7     # Visualize true and predicted values
8     plt.figure(figsize=(8,6))
9     plt.scatter(x_test, y_test, color='purple', s=2, label='true values')
10    plt.scatter(x_test, y_sample_of_predictions, color='deeppink', s=2,
11               label='predicted values')
12    plt.legend()
13    #plt.show()
```

And we get the following results:

$$RMSE = 0.636$$

$$MAE = 0.813$$

Chapter 8

COMPARISON

The main difference between classical statistics and Bayesian inference is that classical statistics treats the parameters of a model as fixed unknowns, while Bayesian inference treats them as random variables with prior distributions. This difference has implications for how simple linear regression is done in each approach.

Classical statistics

In classical statistics, simple linear regression is done by fitting the model to the data using the least squares method. This method minimizes the sum of the squared residuals, which are the differences between the observed values and the predicted values of the response variable. The least squares estimates of the parameters are the values that minimize the sum of the squared residuals.

Bayesian inference

In Bayesian inference, simple linear regression is done by first specifying a prior distribution for the parameters of the model. This prior distribution reflects our beliefs about the parameters before we see the data. Then, we use Bayes' theorem to update our beliefs about the parameters after seeing the data. The posterior distribution of the parameters is a combination of our prior beliefs and the information in the data.

The posterior distribution of the parameters can be used to make inferences about the parameters, such as calculating confidence intervals or credible intervals. It can also be used to make predictions about new data.

Which approach is better for simple linear regression depends on the specific application. Classical statistics is often preferred because it is more straightforward and easier to implement. However, Bayesian inference can be more flexible and can allow for more nuanced inferences.

In general, Bayesian inference is becoming increasingly popular in machine learning and other areas of statistics. This is because it can provide more robust and interpretable results than classical statistics.

Now, we will calculate MAE and RMSE using least squares from classical statistics.

We will use `sklearn` library to fit the line to the train data in classical

(frequentist) fashion using least squares and evaluate the model using MAE and RMSE metrics:

```
1     from sklearn.linear_model import LinearRegression
2     from sklearn.metrics import mean_squared_error, mean_absolute_error
3
4
5     # Comparison to the classical statistics
6     model = LinearRegression()
7     model.fit(x_train.reshape(-1, 1), y_train)
8     y_pred = model.predict(x_test.reshape(-1, 1))
9
10    rmse_cl = np.sqrt(mean_squared_error(y_test, y_pred))
11    mae_cl = mean_absolute_error(y_test, y_pred)
12
13    print(f"Classical RMSE: {rmse_cl}")
14    print(f"Classical MAE: {mae_cl}")
15
```

And we get the following results:

$$RMSE = 0.58$$

$$MAE = 0.43$$